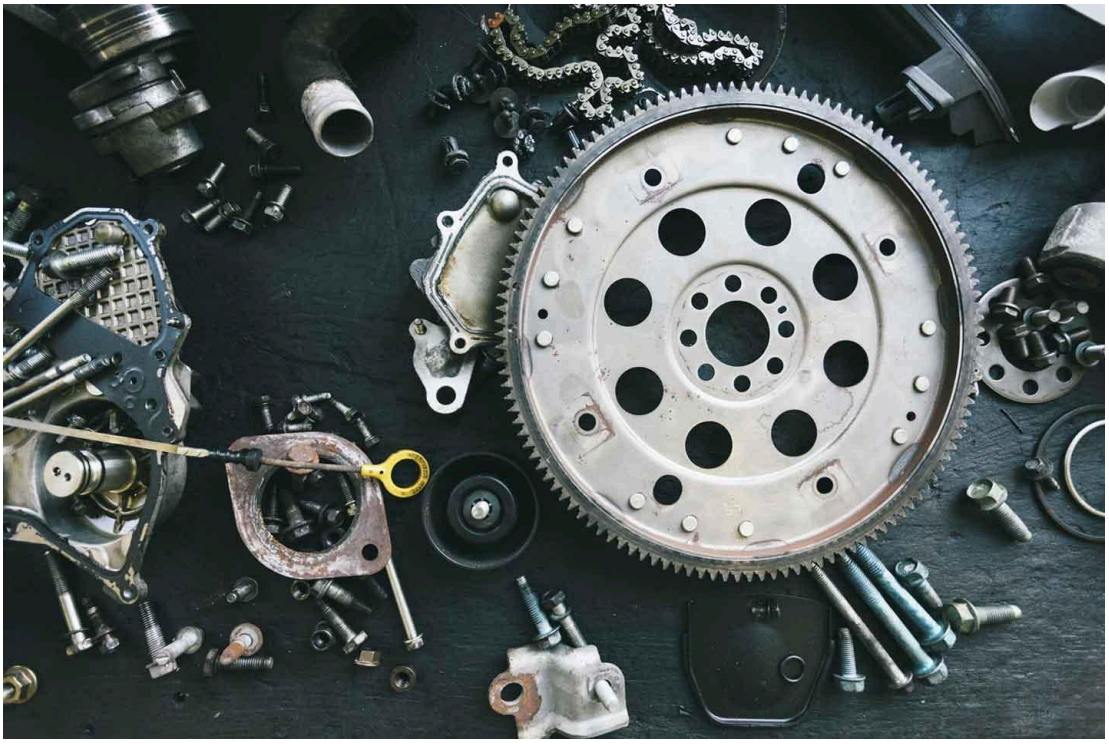
Tema 1

Introducción a Spring Boot

Digitech
FP OFICIAL DIGITAL & TECH



Francisco Belda

ÍNDICE

Teoría. Arquitectura y Fundamentos de Spring **3**
Del ecosistema tradicional a Spring Boot 3
Inversión de Control (IoC) y Inyección de Dependencias (DI) 3
Paso a Paso: Creación y Configuración del Proyecto **5**
Generación del proyecto con Spring Initializr 5
Anatomía del proyecto y análisis del pom.xml 6
Práctica Guiada: Primer Endpoint REST **7**
La clase principal con @SpringBootApplication 7
Explicación de la anotación 7
Creación de un Controlador REST básico 8
Desglose técnico del código anterior: 8
Ejecución y Pruebas 9
Resumen y Ejercicios **10**
Conceptos Clave Recordados 10
Ejercicios 10

Teoría. Arquitectura y Fundamentos de Spring

Del ecosistema tradicional a Spring Boot

Durante años, el desarrollo de aplicaciones empresariales en Java requería una cantidad ingente de código de configuración (archivos XML kilométricos, despliegues manuales de archivos WAR en servidores de aplicaciones externos como Tomcat o JBoss). Spring Framework nació para simplificar esto, pero con el tiempo, configurar Spring también se volvió complejo debido a la gran cantidad de librerías y parámetros requeridos.

Spring Boot surge para solucionar este problema aplicando el principio de "convención sobre configuración":

- **Autoconfiguración**: Spring Boot analiza las librerías (dependencias) que has añadido a tu proyecto en el pom.xml y configura automáticamente los beans necesarios para que la aplicación funcione.
- **Servidor Embebido**: Ya no necesitas instalar un servidor de aplicaciones externo. Spring Boot incluye servidores web embebidos (por defecto Tomcat), permitiendo empaquetar la aplicación en un archivo ejecutable JAR con un simple java -jar.
- **Starters**: Son descriptores de dependencias preempaquetados que agrupan todo lo necesario para una tecnología concreta (por ejemplo, spring-boot-starter-web).

Inversión de Control (IoC) y Inyección de Dependencias (DI)

En la programación orientada a objetos tradicional en Java, cuando una clase necesita utilizar los servicios de otra, suele instanciarla directamente utilizando el operador new:

Programación tradicional

```
class Coche {
    private Motor motor = new MotorGasolina(); // Acoplamiento fuerte
}
```

Este enfoque genera un acoplamiento fuerte: si queremos cambiar MotorGasolina por MotorElectrico, debemos modificar el código fuente de la clase Coche.

Con Inversión de Control (IoC), el flujo de control se invierte: ya no es el programador (ni la clase) quien controla la creación de los objetos, sino un contenedor centralizado (el Contenedor IoC de Spring). A través de la Inyección de Dependencias (DI), es el contenedor el que "inyecta" o proporciona las dependencias que una clase necesita en tiempo de ejecución:

Enfoque Spring (IoC / DI)

```
@Component
class Coche {
    private final Motor motor;

    @Autowired // Inyección a través del constructor
    public Coche(Motor motor) {
        this.motor = motor;
    }
}
```

Paso a Paso: Creación y Configuración del Proyecto

Generación del proyecto con Spring Initializr

Para acelerar la creación de proyectos Spring Boot, utilizaremos la herramienta oficial web Spring Initializr. Configura los siguientes parámetros en la interfaz web:

Parámetro	Valor Seleccionado	Descripción
Project	Maven Project	Gestor de dependencias y ciclo de vida del proyecto.
Language	Java	Lenguaje de programación base.
Spring Boot	Versión estable más reciente (ej. 3.x.x)	Versión del framework.
Group	com.educacion	Identificador de la organización o paquete raíz.
Artifact	demo-spring	Nombre del proyecto/artefacto.
Packaging	Jar	Formato de empaquetado ejecutable.
Java	17 o 21	Versión de la máquina virtual de Java (LTS).

A continuación, haz clic en el botón Add Dependencies y añade las siguientes:

1. **Spring Web**: Incluye herramientas para desarrollar aplicaciones web, incluyendo RESTful, utilizando Spring MVC y Tomcat embebido.
2. **Spring Boot DevTools**: Proporciona recargas automáticas en caliente (*live reload*) cada vez que modificamos el código fuente, agilizando el desarrollo.
3. **Lombok**: librería de Java para mediante anotaciones, reducir el código duplicado.

Haz clic en Generate para descargar un archivo ZIP, descomprímelo y ábrelo desde tu entorno de desarrollo (por ejemplo, IntelliJ IDEA o WebStorm/Eclipse).

ANATOMÍA DEL PROYECTO Y ANÁLISIS DEL POM.XML

Una vez abierto el proyecto en tu IDE, observa la estructura de directorios estándar:

src/main/java/com/educacion/demospring/: Contiene el código fuente en Java y la clase principal de arranque.

src/main/resources/: Contiene los archivos de configuración (como application.properties) y recursos estáticos.

pom.xml: Archivo de configuración de Maven que gestiona las dependencias y plugins del proyecto.

Analicemos la estructura básica del archivo pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>3.2.x</version>
<relativePath/> <!-- lookup parent from repository -->
<لیت/parent>

<groupId>com.educacion</groupId>
<artifactId>demo-spring</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>demo-spring</name>
<description>Proyecto inicial de Spring Boot</description>

<properties>
   <java.version>17</java.version>
</properties>

<dependencies>
   <!-- Dependencia para aplicaciones web / REST -->
   <dependency>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-web</artifactId>
   </dependency>

   <!-- Dependencia para pruebas automáticas -->
   <dependency>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-test</artifactId>
       <scope>test</scope>
   </dependency>
</dependencies>
</project>
```

PRÁCTICA GUIADA: PRIMER ENDPOINT REST

LA CLASE PRINCIPAL CON @SPRINGBOOTAPPLICATION

Dirígete al paquete principal:

src/main/java/com/educacion/demospring/DemoSpringApplication.java.

Esta clase es el punto de entrada de nuestra aplicación:

```
package com.educacion.demospring;


import org.springframework.boot.SpringApplication;

import
org.springframework.boot.autoconfigure.SpringBootApplication;


@SpringBootApplication

public class DemoSpringApplication {


   public static void main(String[] args) {

            SpringApplication.run(DemoSpringApplication.class,
args);

   }

}
```

Explicación de la anotación

`@SpringBootApplication`: Es una anotación compuesta que agrupa tres características fundamentales:

`@Configuration`: Permite registrar beans adicionales en el contexto de Spring o importar clases de configuración.

`@EnableAutoConfiguration`: Le indica a Spring Boot que comience a añadir beans basándose en el classpath, las otras beans y las propiedades configuradas.

`@ComponentScan`: Le indica a Spring que escanee el paquete actual y sus subpaquetes en busca de componentes (como controladores, servicios, repositorios).

Creación de un Controlador REST básico

Crea un nuevo paquete llamado **controller** dentro de **com.educacion.demospring** y dentro de él una clase llamada **SaludoController.java**:

```
package com.educacion.demospring.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import
org.springframework.web.bind.annotation.RestController;

@RestController
public class SaludoController {

   @GetMapping("/saludo")
   public String saludar(@RequestParam(value = "nombre",
defaultValue = "Mundo") String nombre) {
       return "¡Hola, " + nombre + "! Bienvenido al curso de
Spring Boot para Grado Superior.";
   }
}
```

Desglose técnico del código anterior:

- **`@RestController`**: Indica a Spring que esta clase es un controlador web donde cada método devuelve un objeto directamente serializado en formato JSON (o texto plano), en lugar de renderizar una vista HTML tradicional.
- **`@GetMapping("/saludo")`**: Mapea las peticiones HTTP GET que lleguen a la ruta /saludo para que sean atendidas por este método.
- **`@RequestParam`**: Extrae parámetros de la URL (por ejemplo: ?nombre=Ana). Si no se proporciona ninguno, asigna por defecto el valor "Mundo".

Ejecución y Pruebas

Para arrancar la aplicación, puedes ejecutar el siguiente comando en la terminal integrada de tu IDE utilizando Maven:

mvn spring-boot:run

Observa la consola: verás los logs de arranque de Spring Boot y cómo se inicializa el servidor embebido **Tomcat en el puerto 8080**.

Para comprobar que funciona correctamente, abre tu navegador web o herramienta de peticiones (como Postman o cURL) y accede a:

http://localhost:8080/saludo (Respuesta esperada: ¡Hola, Mundo! Bienvenido al curso de Spring Boot para Grado Superior.)

http://localhost:8080/saludo?nombre=Francisco (Respuesta esperada: ¡Hola, Francisco! Bienvenido al curso de Spring Boot para Grado Superior.)

Resumen y Ejercicios

Conceptos Clave Recordados

- Spring Boot automatiza la configuración inicial y despliegue gracias a los starters y servidores embebidos.
- El contenedor IoC administra el ciclo de vida de los objetos, reduciendo el acoplamiento mediante la Inyección de Dependencias.
- Los controladores REST con @RestController permiten exponer endpoints de manera limpia y rápida mediante anotaciones como @GetMapping.

Ejercicios

1. **Modificación del Endpoint**: Añade un nuevo método en SaludoController mapeado a la ruta /info que devuelva un objeto Java plano (DTO) con los datos del curso, el centro educativo y el profesor, verificando cómo Spring Boot lo transforma automáticamente a JSON.
2. **Parámetros Múltiples**: Crea un endpoint /suma que reciba dos parámetros numéricos enteros a través de la URL (a y b) y devuelva el resultado de su suma.